

1. Requirement Analysis

1.1 Project Overview

The system will be a cross-platform desktop application for **macOS and Windows** that allows users to import, download, organize, manage, and consume online media that they are authorized to download.

The application will use a local media-processing engine such as `yt-dlp` together with `FFmpeg` to handle supported online media sources.

The long-term vision is to evolve the application from a simple media downloader into a **personal offline media and learning library**, supporting features such as progress tracking, transcripts, search, notes, AI summaries, quizzes, flashcards, and course-level Q&A.

The initial release will focus primarily on:

Import → Analyze → Download → Organize → Watch

2. Product Goals

The application should:

1. Provide a simple graphical alternative to command-line tools such as `yt-dlp`.
 2. Support individual videos and collections/playlists from compatible sources.
 3. Download media directly to the user's computer.
 4. Automatically organize downloaded media into a local library.
 5. Allow users to watch downloaded content within the application.
 6. Track viewing progress.
 7. Provide a reliable download queue with progress and error handling.
 8. Work consistently across macOS and Windows.
 9. Remain provider-independent so the application is not architecturally tied exclusively to YouTube.
 10. Provide a foundation for future learning and AI functionality.
-

3. Target Users

Primary target users include:

- Students
 - Software developers
 - Certification candidates
 - University students
 - Researchers
 - Content creators
 - Users maintaining offline learning libraries
 - Users with limited or unreliable internet connectivity
 - Users who want to organize media they are permitted to download
-

4. User Personas

4.1 Student / Certification Learner

Wants to download a long tutorial or course playlist and study it offline.

Typical needs:

- Playlist downloads
 - Correct video ordering
 - Progress tracking
 - Resume watching
 - Search and learning tools in future releases
-

4.2 Developer / Technical Learner

Frequently follows tutorials and technical video series.

Typical needs:

- High-quality downloads
 - Playlist management
 - Organized course libraries
 - External SSD support
 - Fast searching
 - Future transcript and AI functionality
-

4.3 Content Creator

Wants to maintain local copies of their own online content.

Typical needs:

- High-quality downloads
 - Metadata preservation
 - Thumbnails
 - Playlist/channel organization
 - Automatic synchronization in future versions
-

5. Supported Platforms

V1 shall support:

- macOS Apple Silicon
- macOS Intel
- Windows x64

Linux is outside the initial release scope but the architecture should not prevent future Linux support.

6. High-Level User Flow

The main application flow shall be:

```
graph TD; A[Launch Application] --> B[Paste Media URL]; B --> C[Analyze URL]; C --> D[Display Media / Playlist Information]; D --> E[Select Items]; E --> F[Select Quality / Format]; F --> G[Choose Download Location]; G --> H[ ];
```

Add to Download Queue



Download Media



Post-process Media



Add Media to Library



Watch Media



Track Viewing Progress

7. System Scope

7.1 Included in V1

V1 will include:

- URL analysis
- Supported source detection
- Single video downloads
- Playlist/collection downloads
- Video selection within playlists
- Quality selection
- Format selection
- Download queue
- Download progress
- Concurrent downloads
- Pause/resume
- Cancel download
- Retry failed downloads
- Local media library
- Collection organization
- Local media player
- Watch-progress tracking
- Configurable download directory
- External storage support
- Download history
- Application settings
- macOS packaging
- Windows packaging

8. Out of Scope for V1

The following features will not be required for the initial release:

- Mobile applications
- Cloud video storage
- Media sharing between users
- Social features
- DRM circumvention
- Account sharing
- AI summaries
- AI question answering
- Flashcards
- Quiz generation
- Transcript generation
- Semantic search
- Browser extension
- Automatic playlist synchronization
- Cross-device synchronization
- User-generated public libraries

These may be introduced in future versions.

9. Core Functional Requirements

FR-001 — URL Input

The system shall allow the user to paste a media URL into the application.

The URL input shall:

- accept supported HTTP/HTTPS URLs
 - trim accidental whitespace
 - validate basic URL structure
 - reject invalid input before analysis
-

FR-002 — URL Analysis

The user shall be able to request analysis of a supplied URL.

The system shall use the configured media engine to determine:

- whether the source is supported
- source/provider
- media title
- media type
- thumbnail
- duration where available
- available media items
- playlist information where applicable

The analysis shall occur without downloading the complete media file.

FR-003 — Supported Source Detection

The system shall automatically determine the media source using the media engine.

Examples may include:

- YouTube
- Vimeo
- supported social media sources
- supported video websites
- direct media URLs
- other sources supported by the underlying media engine

The UI shall not depend on provider-specific data structures.

FR-004 — Unsupported Source Handling

If a URL cannot be processed, the system shall display a clear error.

Example:

This media source is currently unsupported.

The application shall not crash because of unsupported URLs.

10. Media Metadata Requirements

FR-005 — Media Preview

For a single media item, the application shall display:

- title
 - thumbnail where available
 - duration
 - source/provider
 - available quality information where available
-

FR-006 — Playlist Detection

If the supplied URL represents a playlist or collection, the application shall detect it automatically.

The system shall display:

- playlist title
 - playlist thumbnail where available
 - number of items
 - individual media items
 - media duration where available
 - playlist index/order
-

FR-007 — Playlist Item Selection

Users shall be able to:

- select all items
- deselect all items
- select individual items
- exclude individual items

Only selected items shall be added to the download queue.

11. Download Configuration

FR-008 — Quality Selection

The application shall allow users to select a preferred download quality.

Initial options shall include:

Best Available
2160p / 4K
1440p
1080p
720p
480p
Audio Only

Options unavailable for the selected media may be hidden or disabled.

FR-009 — Output Format

Users shall be able to choose an output format where technically supported.

Initial formats:

MP4
MKV
WebM
M4A
MP3

FR-010 — Default Download Settings

Users shall be able to define default preferences for:

- video quality
- output format

- download location
- concurrent downloads

These settings shall persist between application sessions.

12. Download Queue

FR-011 — Add Download

After configuring a media item or playlist, the user shall be able to add selected media to the download queue.

FR-012 — Queue Management

The system shall maintain a download queue.

Each download shall have one of the following states:

Queued
Analyzing
Downloading
Post-processing
Paused
Completed
Failed
Cancelled

FR-013 — Download Progress

For active downloads, the application should display where available:

- filename
- percentage completed
- downloaded size
- total size
- download speed
- estimated remaining time
- current processing state

FR-014 – Concurrent Downloads

The system shall support multiple simultaneous downloads.

Users shall be able to configure the maximum concurrency.

Example:

```
1 download
2 downloads
3 downloads
4 downloads
```

The Queue Manager shall ensure that the concurrency limit is respected.

FR-015 – Pause Download

Users shall be able to pause an active download.

The system shall preserve partial download data when supported.

FR-016 – Resume Download

Users shall be able to resume a previously paused or interrupted download.

The application should continue from the existing partial file instead of restarting where supported.

FR-017 – Cancel Download

Users shall be able to cancel an active or queued download.

The system shall ask whether temporary partial files should be kept or removed where appropriate.

FR-018 – Retry Download

Failed downloads shall provide a retry option.

Retrying shall not create duplicate media-library records.

13. Error Handling

FR-019 — Download Failure Handling

If a download fails, the application shall:

- mark the download as failed
 - preserve an appropriate error description
 - keep the application operational
 - provide a retry option
-

FR-020 — Internet Failure Handling

If internet connectivity is interrupted during a download, the application shall preserve recoverable download state whenever possible.

FR-021 — Storage Error Handling

The application shall detect common storage errors including:

- insufficient disk space
- missing external drive
- inaccessible directory
- filesystem permission failure

The user shall receive a clear message rather than a generic application error.

14. File Management

FR-022 — Download Directory

Users shall be able to choose where downloaded media is stored.

Example:

```
~/Videos/AppName/
```

The application shall also support:

- external SSDs
- external HDDs
- user-selected folders

FR-023 — Playlist Folder Structure

Playlist downloads should be organized into individual folders.

Example:

```
Media Library/
```

```
AWS Solutions Architect/
```

- 01 - Introduction.mp4
- 02 - IAM.mp4
- 03 - EC2.mp4

```
React Advanced/
```

- 01 - Components.mp4
- 02 - Hooks.mp4

FR-024 — Ordered Filenames

Playlist items shall retain playlist order.

Example:

- 01 - Introduction.mp4
 - 02 - IAM Basics.mp4
 - 03 - IAM Roles.mp4
-

FR-025 — Filename Sanitization

The application shall remove or replace characters that are invalid for the current operating system.

File naming behavior must work correctly on both macOS and Windows.

FR-026 — Existing File Detection

Before downloading media, the system should determine whether the item has already been successfully downloaded.

Users should not unintentionally download identical media multiple times.

15. Local Media Library

FR-027 — Library Creation

Successfully downloaded media shall automatically be registered in the local application library.

FR-028 — Collections

Media belonging to the same playlist or course shall be grouped into a collection.

Example:

My Library

AWS Solutions Architect
42 videos

Claude Certification
28 videos

Advanced React
17 videos

FR-029 — Collection Details

Opening a collection shall display:

- collection title
 - thumbnail
 - number of media items
 - viewing progress
 - individual items
 - item status
-

FR-030 — Library Search

V1 shall allow users to search the library by media title or collection title.

Transcript/content search will be introduced in a future version.

FR-031 — Remove Library Item

Users shall be able to remove an item from the library.

The system should allow the user to choose between:

Remove from library only

or

Remove from library and delete local file

FR-032 — Remove Collection

Users shall be able to remove an entire collection.

The same distinction shall exist between removing database records and deleting physical files.

16. Local Media Player

FR-033 — Playback

Users shall be able to play supported downloaded video and audio files directly within the application.

FR-034 — Standard Playback Controls

The player shall provide:

- play
 - pause
 - seek
 - volume
 - mute
 - fullscreen
 - playback position
 - total duration
-

FR-035 — Watch Progress

The system shall periodically save the user's playback position.

Example:

Current position:
37:14 / 52:41

FR-036 — Resume Watching

When a user returns to partially watched media, the application shall allow playback to continue from the previous position.

FR-037 — Completion Tracking

The system shall mark media as completed after the user reaches a configurable completion threshold.

Suggested initial threshold:

90%

17. Continue Watching

FR-038 — Continue Watching Section

The home or library screen shall provide a section containing recently viewed incomplete media.

Example:

Continue Watching

EC2 Auto Scaling

24:31 / 46:02

Claude Architecture

12:04 / 37:49

18. Library Progress

FR-039 — Collection Progress

The application shall calculate viewing progress for each collection.

Example:

AWS Solutions Architect

27 / 42 completed

19. Settings

FR-040 – General Settings

The application shall provide a Settings screen.

FR-041 – Download Settings

Users shall be able to configure:

- download directory
 - default quality
 - default format
 - concurrent downloads
 - filename style
-

FR-042 – Library Settings

Users shall be able to configure the location of the local media library.

FR-043 – Storage Information

The application should display how much storage the media library currently consumes.

Example:

Media Library

47.2 GB used

20. Media Engine Management

FR-044 — Bundled Media Engine

The application shall include the required media download engine so users do not need to manually install `yt-dlp`.

FR-045 — Bundled Media Processing

Required FFmpeg components shall be distributed or installed as part of the application deployment process.

Users should not need to configure FFmpeg manually.

FR-046 — Media Engine Version

The application shall be able to determine the currently installed media-engine version.

Future releases should support controlled media-engine updates independently of the Electron application.

21. Application Updates

FR-047 — Update Detection

The application should be capable of checking whether a newer application version is available.

FR-048 — Update Notification

When an update is available, the user shall receive a notification containing:

- current version
- available version

- update action

22. Platform Integration

FR-049 — macOS Support

The application shall provide a properly packaged macOS application.

Target architectures:

Apple Silicon
Intel

FR-050 — Windows Support

The application shall provide a Windows installer supporting Windows x64.

23. Privacy Requirements

FR-051 — Local-First Operation

Downloads, library metadata, and playback history shall remain on the user's device by default.

FR-052 — No Media Upload

Downloaded video/audio files shall not be uploaded to application servers.

FR-053 — Authentication Data

If authentication/cookie support is introduced later, credentials and cookies shall remain local unless the user explicitly authorizes another behavior.

24. Content Protection Requirement

FR-054 — DRM-Protected Content

The application shall not intentionally circumvent DRM systems.

DRM-protected media may be reported as unsupported.

25. V1 Primary Screens

The initial application should contain the following screens:

```
graph TD; Home --> AnalyzeMedia[Analyze Media]; AnalyzeMedia --> DownloadConfiguration[Download Configuration]; DownloadConfiguration --> Downloads; Downloads --- Active; Downloads --- Queued; Downloads --- Completed; Downloads --- Failed; Downloads --> Library; Library --> Collection; Collection --> MediaPlayer[Media Player]; MediaPlayer --> Settings;
```

Home
↓
Analyze Media
↓
Download Configuration

Downloads
├── Active
├── Queued
├── Completed
└── Failed

Library
↓
Collection
↓
Media Player

Settings

26. Future Functional Requirements

The architecture should allow future implementation of the following without major redesign.

Playlist Synchronization

Automatically detect newly added media in an existing collection.

Browser Extension

Allow supported browser pages to be sent directly to the desktop application.

Subtitle Support

Download, embed, or display available subtitles.

Transcript Generation

Create searchable transcripts from downloaded content.

Transcript Search

Search across spoken content rather than only media titles.

Example:

Search:

"security groups versus NACL "

Result:

AWS Networking

Video 14

09:24

[Jump to timestamp]

Notes

Allow users to attach timestamped notes to media.

Bookmarks

Allow users to bookmark specific timestamps.

AI Summaries

Generate summaries for individual media items.

Course-Level AI Q&A

Allow users to ask questions against an entire collection.

Flashcards

Automatically generate study flashcards from transcripts.

Quiz Generation

Generate quizzes from downloaded learning material.

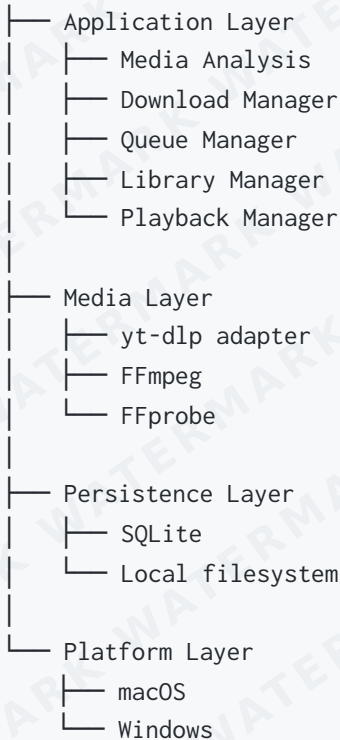
Local AI

Potential future support for local inference platforms such as Ollama.

27. Major System Components

The system shall be divided into the following major components:

```
Electron Application
|
├── Presentation Layer
|   └── React UI
```



28. V1 Success Criteria

The MVP shall be considered successful when a user can:

1. Install the application without separately installing technical dependencies.
2. Paste a supported media URL.
3. Analyze the URL.
4. Preview a video or playlist.
5. Select which media items to download.
6. Select video quality.
7. Download the selected media.
8. View download progress.
9. Pause, resume, retry, or cancel downloads.
10. Find completed media automatically organized in the library.
11. Play downloaded media within the application.
12. Close and reopen the application.
13. Continue watching from the previous playback position.
14. Use the same core workflow on both macOS and Windows.

29. Core V1 Principle

The first version should solve one problem exceptionally well:

Allow users to import authorized online media, reliably download it, organize it into a structured offline library, and continue consuming it later.

Features unrelated to this core workflow should not delay V1.